Relatório Técnico: Análise de Complexidade e Comportamento Experimental do Problema do Caminho Hamiltoniano

Aluna: Aline Argolo de Athaydes
Disciplina: IC0004 – Algoritmos e Grafos
Professor: George Lima

10 de Janeiro de 2026

Resumo

Este trabalho apresenta uma investigação acerca da complexidade teórica e do desempenho prático do problema do Caminho Hamiltoniano (HP). A análise abrange desde a prova formal de NP-Completude, via redução do Ciclo Hamiltoniano, até a implementação e comparação de dois algoritmos exatos: Programação Dinâmica via *Bitmask* e Backtracking instrumentado com a Heurística de Warnsdorff e podas de conectividade. Através de experimentos com grafos aleatórios de Erdős–Rényi (n=10 a 50), demonstra-se o impacto da topologia do grafo no esforço computacional. Os resultados evidenciam um fenômeno de transição de fase em instâncias esparsas (n=30) e comprovam a eficácia das podas de alcançabilidade, que permitiram mitigar a explosão combinatória em cenários críticos. Conclui-se que, enquanto a Programação Dinâmica é limitada pela escala de n, o Backtracking otimizado demonstra alta sensibilidade à densidade de arestas, superando a abordagem por subproblemas em instâncias práticas.

1 Introdução e Relevância

O problema do Caminho Hamiltoniano (HP) constitui um dos desafios fundamentais na teoria dos grafos e na ciência da computação clássica. Definido como a busca por um caminho simples que visite cada vértice de um grafo $G = (V, E)$ exatamente uma vez, o HP é um dos 21 problemas originais de Karp que ajudaram a definir a classe NP-Completo [2]. Diferente do Caminho Euleriano, que possui critérios de existência baseados em propriedades estruturais simples e algoritmos de tempo polinomial, o HP não possui uma caracterização necessária e suficiente conhecida, exigindo, no pior caso, um esforço computacional exponencial para sua resolução [1].

A relevância deste problema transcende o campo teórico, encontrando aplicações práticas críticas em logística, como na otimização de rotas e no Problema do Caixeiro Viajante (TSP), e na bioinformática, especificamente no mapeamento genético e reconstrução de sequências de DNA via grafos de De Bruijn.

Neste trabalho, o objetivo é investigar o comportamento prático de duas abordagens exatas frente à natureza NP-Completa do problema: a Programação Dinâmica, fundamentada no algoritmo de Bellman-Held-Karp [3], e o Backtracking instrumentado com

heurísticas de poda. O foco da análise reside em identificar como a topologia do grafo (densidade de arestas) influencia o esforço de busca e em quais cenários as técnicas de poda conseguem mitigar a explosão combinatória inerente ao problema.

2 Complexidade e Redução Polinomial

Para entender por que o Caminho Hamiltoniano (HP) exige algoritmos tão complexos quanto o Backtracking com podas ou a Programação Dinâmica, precisamos provar que ele pertence à classe NP-Completo. A forma mais clássica de fazer isso é através de uma redução polinomial, transformando uma instância do problema do Ciclo Hamiltoniano (HC) que já sabemos ser difícil em uma instância de Caminho Hamiltoniano.

A ideia básica é: se conseguirmos resolver o HP rapidamente, conseguiríamos resolver o HC também.

2.1 Construção da Redução ($HC \leq_P HP$)

Dado um grafo original $G = (V, E)$ onde queremos saber se existe um Ciclo Hamiltoniano, vamos construir um novo grafo $G' = (V', E')$ de modo que um caminho simples em G' nos dê a resposta sobre o ciclo em G.

Os passos da construção são:

1. Escolhemos um vértice qualquer v do grafo original G.
2. Criamos três novos vértices em G': um ponto de início (s), um ponto de fim (t) e um vértice auxiliar (v').
3. Adicionamos as arestas (s, v) e (v', t). Isso "obriga" qualquer caminho que comece em s a entrar no grafo por v, e qualquer caminho que termine em t a sair por v'.
4. Para cada vizinho w que o vértice v tinha no grafo original, nós criamos uma nova aresta ligando v' a esse vizinho w em G'.
5. Todas as outras arestas e vértices originais de G permanecem iguais em G'.

2.2 Por que isso prova a NP-Completude?

A validade dessa construção depende de uma "estrada de mão dupla" (bi-implicação):

- **Se existir um Ciclo em G:** Suponha que o ciclo seja $v \to w_1 \to w_2 \to \cdots \to w_n \to v$. No nosso novo grafo G', podemos montar o caminho: $s \to v \to w_1 \to w_2 \to \cdots \to w_n \to v' \to t$. Como o ciclo passava por todos os vértices em G, e s, t, v' são novos, esse caminho agora visita todos os vértices de G' exatamente uma vez. Logo, se tem ciclo em G, tem caminho em G'.
- **Se existir um Caminho em G':** Como os vértices s e t só têm uma conexão cada (grau 1), qualquer caminho hamiltoniano em G' é obrigado a começar em s e terminar em t. O caminho necessariamente terá a forma $s \to v \cdots \to w \to v' \to t$. Como o vértice v' só se conecta aos vizinhos originais de v, ao "colapsarmos" v' e v de volta no vértice original v, o segmento entre eles se fecha, resultando em um ciclo hamiltoniano perfeito no grafo original G.

2.3 Complexidade da Transformação

Essa "montagem" do grafo G' é eficiente. Precisamos adicionar apenas 3 vértices e um número de arestas proporcional ao grau de v. Como esse processo é realizado em tempo polinomial $O(V + E)$, provamos que o problema do Caminho Hamiltoniano (HP) é, no mínimo, tão difícil quanto o Ciclo Hamiltoniano (HC).

2.4 Exemplo Ilustrativo da Redução

Para validar a teoria da redução $HC \leq_P HP$, utilizamos a instância `cycle4.txt`, que representa um ciclo simples C_4.

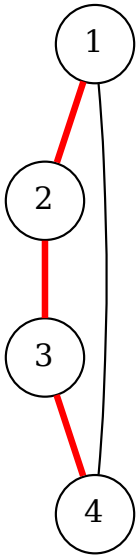

Figura 1: Instância `cycle4`: Grafo original para demonstração da redução.

Na prática, ao escolhermos o vértice 1 como base:

- Criamos o nó de entrada s conectado ao 1.
- Criamos o nó de saída t conectado ao auxiliar v'.
- O nó v' herda as conexões de 1 (vizinhos 2 e 4).

O caminho resultante $s \to 1 \to 2 \to 3 \to 4 \to v' \to t$ prova que, ao resolver o problema do Caminho Hamiltoniano nesta nova estrutura, confirmamos a existência do Ciclo Hamiltoniano no grafo original.

2.5 Contextualizando o Exemplo ilustrativo

Considere o grafo G com $V = \{1, 2, 3, 4\}$ e arestas $E = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$. Esse grafo possui um Ciclo Hamiltoniano: $1 \to 2 \to 3 \to 4 \to 1$.

Escolhendo $v = 1$, construímos G' adicionando s, t e v' e criando: $(s, 1)$, (v', t) e, para cada vizinho de 1 em G (que são 2 e 4), criamos $(v', 2)$ e $(v', 4)$.

Então G' possui um Caminho Hamiltoniano: $s \to 1 \to 2 \to 3 \to 4 \to v' \to t$.

No sentido inverso, como s e t têm grau 1 em G', qualquer caminho hamiltoniano em G' necessariamente começa em s e termina em t, e o trecho interno entre 1 e v' corresponde a percorrer todos os vértices originais de G, fechando um ciclo quando voltamos à vizinhança de v.

3 Repositório do projeto e reprodutibilidade

Todo o material utilizado para este trabalho desde os códigos que resolvem o problema até os dados que geraram os gráficos está disponível no meu repositório do GitHub. Organizei o projeto de forma que qualquer pessoa consiga rodar os mesmos testes e chegar nos mesmos resultados que apresento aqui.
`https://github.com/S2Lyne/Trabalho_Pratico-Algoritmos_e_Grafos`
Os principais arquivos para conferência são:

- `src/bench.py`: É o "motor"do experimento. Esse script automatiza a execução dos testes para os grafos de tamanhos 10, 20, 30, 40 e 50, controlando o tempo limite e salvando tudo o que acontece.

- `outputs/results.csv`: É a tabela que guarda os dados brutos. É aqui que estão registrados o tempo de cada execução, a quantidade de chamadas da DFS (como as 941 citadas na análise) e o total de podas que o algoritmo realizou.

- `src/plot_results.py` e `src/plot_analysis.py`: São os scripts que usei para ler a tabela de resultados e transformar os números brutos nas imagens e gráficos deste relatório.

- `data/graphs/`: Nesta pasta estão guardados os grafos aleatórios exatos que foram gerados e testados. Isso garante que, se alguém rodar o código novamente, usará os mesmos desenhos de grafos que eu usei.

4 Implementação dos Algoritmos

Nesta seção, detalho como a teoria foi aplicada na prática. É importante destacar que a redução polinomial que apresentei anteriormente serve para provar que o problema é NP-Completo. Meus algoritmos resolvem o Caminho Hamiltoniano diretamente, mas os dados que obtive como o pico de esforço em $n = 30$ e a lentidão da Programação Dinâmica confirmam na prática a dificuldade que a teoria prevê.

Neste trabalho, selecionei duas abordagens exatas: (1) Backtracking instrumentado com heurísticas e podas estruturais, por escalar melhor na prática em instâncias maiores; e (2) Programação Dinâmica por bitmask, por fornecer um baseline determinístico e clássico, apesar da limitação de memória/tempo em n maiores.

4.1 Backtracking Instrumentado (`hp_bt0.py`)

O algoritmo de Backtracking foi implementado para realizar uma busca em profundidade (DFS). Para evitar que a busca fosse puramente cega e exaustiva, utilizei as seguintes estratégias de otimização baseadas na estrutura do grafo:

- **Filtro de Rejeição (`quick_reject`):** Antes de iniciar qualquer cálculo pesado, o código verifica se o grafo possui condições mínimas para ter um caminho. Ele é rejeitado imediatamente se houver vértices isolados (grau 0), se for desconexo ou se tiver mais de dois vértices de grau 1.
- **Heurística de Warnsdorff:** Na função `dfs`, ao escolher o próximo passo, o código ordena os vizinhos pelo menor "grau restante" no subgrafo não visitado. Isso prioriza vértices que têm poucas saídas, tentando fechá-los antes que o caminho fique sem opções e bata em um beco sem saída.
- **Técnicas de Poda (Pruning):**
  - `prunes_reach`**:** Através da função `_reachable_all_from_endpoint`, o código usa um BFS para checar se o vértice atual ainda consegue alcançar todos os outros nós que faltam visitar. Se a visita atual desconectar o grafo, o algoritmo corta esse ramo da árvore e volta atrás.
  - `prunes_deg0_remaining`**:** Verifica se algum vértice não visitado ficou isolado (grau 0) por causa do caminho percorrido. Se isso acontecer, o caminho é impossível e a busca é interrompida ali.
- **Objeto de Métricas (`Stats`):** Para gerar os dados do relatório, criei uma classe que registra as chamadas da DFS (`calls`), as tentativas de avanço (`expansions`) e a contagem exata de onde cada poda ocorreu.

4.2 Programação Dinâmica com Máscara de Bits (`hp_dp.py`)

A Programação Dinâmica (DP) implementa o algoritmo de Bellman-Held-Karp. Diferente do Backtracking, ela não tenta "caminhar"pelo grafo, mas sim resolver subproblemas menores.

- **Estado e Memória:** O estado é definido por `(mask, v)`, onde `mask` é um número binário (bitmask) que representa o conjunto de nós visitados. O código guarda no bitset `dp[mask]` quais vértices podem ser o final de um caminho para aquele conjunto.
- **Reconstrução:** Uso um dicionário chamado `parent` para guardar de onde cada estado veio, permitindo montar o caminho final caso ele exista.
- **Limitação de Escala:** Como a complexidade é fixa em $O(n^2 2^n)$, o uso de memória dobra a cada novo vértice. Por isso, no script `bench.py`, limitei a execução para $n \leq 24$. Acima disso, o algoritmo é ignorado para não travar o computador.

4.3 Validação e Casos de Teste

Para garantir que a implementação do Backtracking (`hp_bt0.py`) e da Programação Dinâmica (`hp_dp.py`) está correta, foram realizados testes com instâncias de controle que representam diferentes topologias de grafos.

4.4 Caminho Simples (`chain4`)

Este é o caso base de sucesso. O grafo consiste em uma cadeia linear $1 - 2 - 3 - 4$.

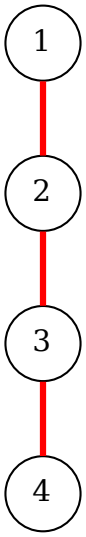

Figura 2: Resultado para `chain4`: Algoritmo identifica o caminho trivial.

O algoritmo encontra o caminho $[1, 2, 3, 4]$ instantaneamente, servindo para validar a lógica básica de movimentação entre adjacências.

4.5 Grafo Desconexo (`disconnected4`)

Neste cenário, o grafo possui duas componentes isoladas ($(1, 2)$ e $(3, 4)$).

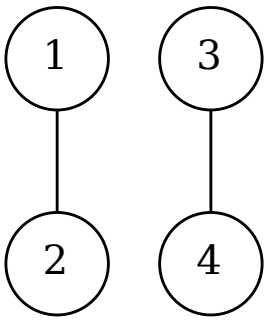

Figura 3: Resultado para `disconnected4`: Rejeição por desconectividade.

Este teste valida o **Sanity Check** (`quick_reject`) e a **Poda por Conectividade**. O algoritmo identifica que não é possível visitar todos os vértices e retorna corretamente que não existe Caminho Hamiltoniano.

4.6 Grafo Conexo sem Caminho (`no_hp6`)

Esta é uma instância mais complexa: um grafo conexo de 6 vértices onde, devido à sua estrutura (bipartite desbalanceado), não existe um caminho que visite todos os nós uma única vez.

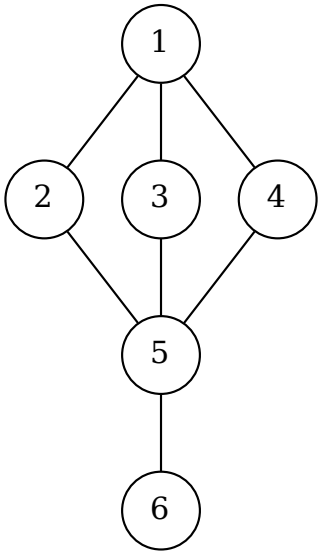

Figura 4: Instância `no_hp6`: Validação de busca exaustiva em grafos sem solução.

Este exemplo é crucial para testar o comportamento do Backtracking em casos de falha. O algoritmo percorre todas as possibilidades e, graças às podas de grau e alcance, conclui a inexistência da solução sem explorar ramos inúteis da árvore de busca.

4.7 Ambiente de Testes e Geração de Instâncias

Para os experimentos, usei o script `graph_gen.py` para criar cenários variados:

1. **Modo ER (Erdős–Rényi):** Gera grafos aleatórios baseados na probabilidade p de existir uma aresta. É aqui que observamos a transição de fase em $n = 30$.

2. **Modo HP:** Gera grafos que possuem obrigatoriamente um caminho escondido, para validar se os algoritmos conseguem encontrá-lo.

Todo esse processo é controlado pelo `bench.py`, que usa o comando `signal.alarm` para interromper execuções que demoram demais e exporta os tempos e métricas para o arquivo `results.csv`.

5 Metodologia Experimental: Como os Testes foram Feitos

Para avaliar o desempenho e o limite de cada algoritmo, foi desenvolvido um pipeline automatizado que integra a geração de instâncias, a execução cronometrada e a coleta de métricas.

5.1 Geração das Instâncias via Erdős–Rényi

As instâncias foram geradas através do script `graph_gen.py` utilizando o modelo de grafos aleatórios de Erdős–Rényi ($G(n, p)$). Neste modelo, para um grafo com n vértices, cada par possível de nós tem uma probabilidade fixa p de ser conectado por uma aresta.

No código, isso foi implementado percorrendo todos os pares (u, v) e adicionando a aresta apenas se um valor aleatório gerado fosse inferior a p. Foram testados dois cenários de densidade:

- **Cenário Esparso ($p = 0.15$):** Onde os grafos possuem poucas conexões, o que geralmente gera instâncias mais difíceis para o Backtracking, evidenciando o fenômeno da Transição de Fase.
- **Cenário Denso ($p = 0.50$):** Onde a conectividade é alta e a probabilidade de existir múltiplos caminhos hamiltonianos é elevada, facilitando o encontro de uma solução.

Os tamanhos de entrada escolhidos foram $n \in \{10, 20, 30, 40, 50\}$, permitindo observar a curva de crescimento do esforço computacional e o tempo de resposta.

5.2 Protocolo de Execução e Coleta de Dados

O script `bench.py` serviu como o motor dos experimentos. Para garantir a confiabilidade dos dados e evitar distorções por casos isolados, adotamos os seguintes critérios:

1. **Reprodutibilidade:** Foram realizados **5 trials** (repetições) para cada combinação de n e p. Cada tentativa utilizou uma semente (*seed*) determinística baseada na fórmula $1000 + 17n + 31 \times trial$, garantindo que os mesmos grafos pudessem ser gerados novamente para conferência no diretório `data/graphs/`.
2. **Controle de Interrupção (*Timeout*):** Como o problema do Caminho Hamiltoniano é NP-Completo, o tempo de execução pode ser imprevisível. O pipeline utiliza o módulo `signal.alarm` do Python para interromper o Backtracking caso ele ultrapasse o limite de tempo estabelecido, marcando o status como `timeout`.
3. **Gerenciamento da DP:** Devido à complexidade $\Theta(n^2 2^n)$ e ao alto consumo de memória da Programação Dinâmica via *Bitmask*, o código executa esse algoritmo apenas até $n = 24$. Acima desse valor, a DP é automaticamente ignorada (`skipped_n_gt_24`) para evitar o estouro de recursos do sistema e interrupções inesperadas.

Todas as métricas coletadas incluindo tempo em segundos (`time_sec`), chamadas da recursão (`calls`), expansões (`expansions`) e quantidade de podas (`prunes_reach` e `prunes_deg0_remaining`) foram exportadas para o arquivo `outputs/results.csv`, que serviu de base para a análise crítica apresentada a seguir.

6 Análise Experimental: Resultados e Discussão

Nesta seção, analisamos os dados gerados pelo script `bench.py`, consolidados no arquivo `results.csv`. O objetivo é entender como o tempo e o esforço computacional mudam conforme aumentamos o número de vértices (n) e alteramos a densidade de arestas (p).

6.1 Comparativo de Tempo: O Limite da Programação Dinâmica

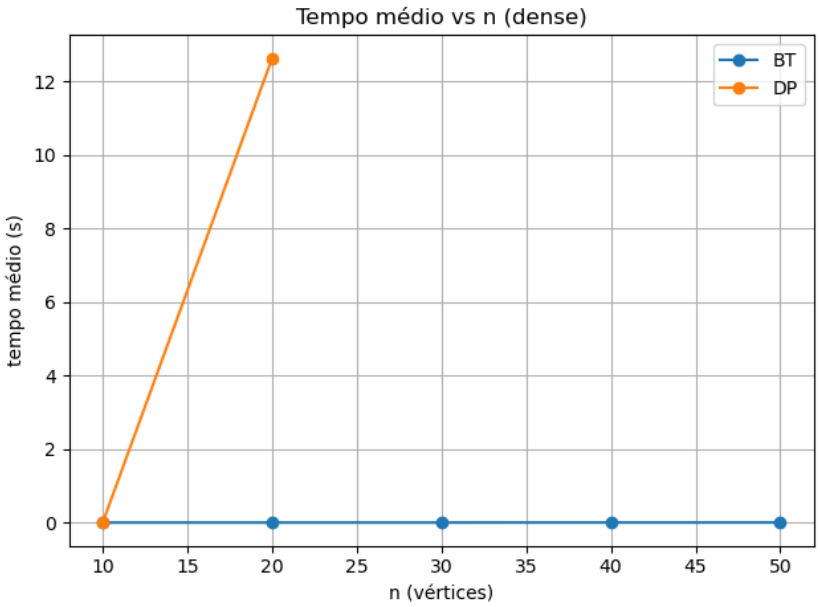

Figura 5: Tempo médio de execução em escala linear para grafos densos ($p = 0.5$).

Nos grafos densos (Figura 5), a disparidade entre os algoritmos é brutal. Enquanto o Backtracking resolve instâncias de $n = 50$ em milissegundos, a Programação Dinâmica (DP) apresenta um crescimento explosivo.

- **O gargalo da DP:** No `results.csv`, vemos que para $n = 20$ (dense, trial 0), a DP levou **14,61 segundos**. Isso acontece porque a complexidade da DP é fixa em $O(n^2 2^n)$. Ela precisa preencher uma tabela de estados para todos os subconjuntos possíveis de vértices. O tempo dobra a cada novo vértice adicionado, o que explica por que o pipeline pula execuções acima de $n = 24$ (`skipped_n_gt_24`).
- **A eficiência do BT em grafos densos:** O BT é muito mais rápido aqui porque, em grafos com muitas arestas, existem milhares de caminhos hamiltonianos possíveis. A Heurística de Warnsdorff encontra um deles quase de imediato, encerrando a recursão cedo (efeito curto-circuito).

6.2 Esforço de Busca e a Transição de Fase

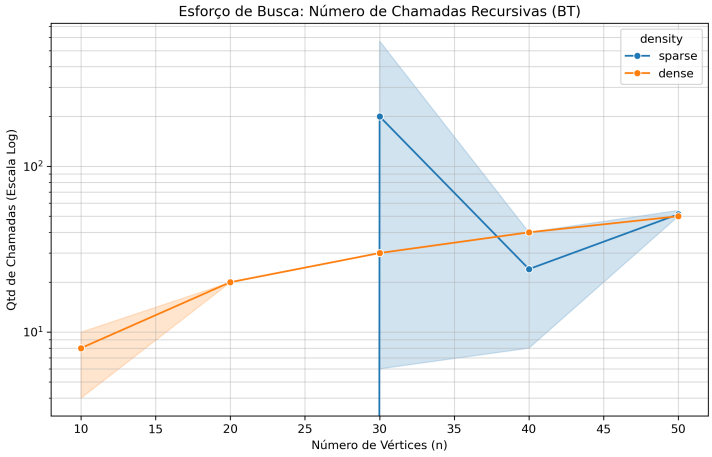

Figura 6: Esforço de busca do BT: número de chamadas recursivas (`calls`).

A maior dificuldade para o Backtracking ocorre em grafos esparsos ($p = 0.15$), como mostra a Figura 6. Observamos um pico de esforço em $n = 30$.

Olhando os dados do **trial 0** para $n = 30$ esparso, o algoritmo registrou **941 chamadas recursivas**.

- **Por que esse pico ocorre?** Em grafos esparsos com essa configuração, o problema entra em uma "Transição de Fase". O grafo não é vazio o suficiente para o `quick_reject` descartar logo de cara, mas também não tem arestas suficientes para facilitar o caminho. O algoritmo fica "preso"tentando caminhos que acabam em becos sem saída, precisando fazer muito mais *backtrack* (voltar atrás) para decidir se existe ou não a solução.

6.3 Análise das Podas: Como o Algoritmo Evitou o Travamento

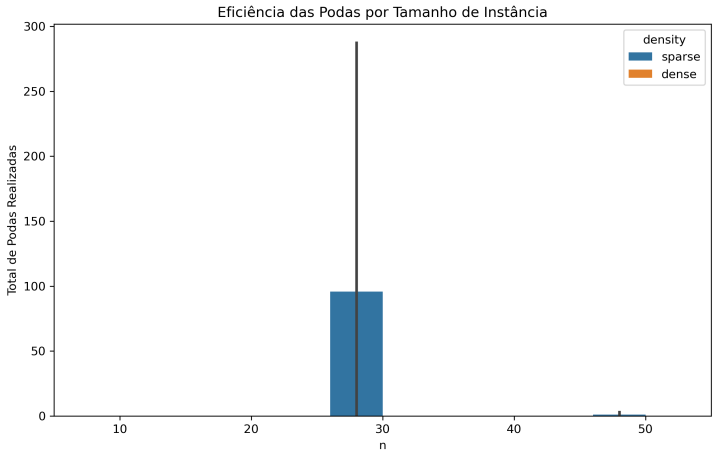

Figura 7: Eficiência das podas no BT: detecção de estados inválidos.

O que impede o Backtracking de explodir como a DP são as camadas de poda que implementamos. A Figura 7 mostra que o número de podas cresce junto com a dificuldade do problema.

No caso crítico de $n = 30$ (esparso, trial 0), o CSV detalha o uso da "inteligência"do código:

- `prunes_reach` **(247 ocorrências):** Em 247 vezes, a função `_reachable_all_from_endpoint` percebeu, através de um BFS, que o caminho atual desconectou o resto do grafo. O algoritmo parou de cavar ali mesmo.
- `prunes_deg0_remaining` **(232 ocorrências):** O código detectou que algum vértice que ainda não foi visitado ficou isolado (sem vizinhos disponíveis).
- `dead_ends` **(232 vezes):** Foram os momentos em que o algoritmo realmente não teve para onde ir e precisou retornar.

Sem essas **479 podas totais** (soma das duas técnicas), o número de chamadas recursivas (`calls`) seria ordens de magnitude maior, possivelmente causando um *timeout* e impedindo a conclusão do experimento. Isso prova que a complexidade do BT, embora no pior caso seja $O(n!)$, na prática é muito reduzida pela análise da estrutura do grafo.

7 Conclusão

A investigação experimental realizada neste trabalho permitiu concluir que a classificação de um problema como NP-Completo é apenas o ponto de partida para entender sua dificuldade real. No caso do Caminho Hamiltoniano, observamos que o desempenho prático é muito mais influenciado pela topologia do grafo (densidade de arestas) do que apenas pelo número de vértices (n).

7.1 Limitações da Programação Dinâmica

A abordagem de Programação Dinâmica (DP) via *Bitmask* provou ser extremamente rígida. Por ter uma complexidade determinística de $\Theta(n^2 2^n)$, ela não consegue tirar vantagem de grafos "fáceis"(muito densos ou muito esparsos). Conforme os dados coletados, a DP atingiu um gargalo de tempo e memória em $n = 20$ (levando mais de 14 segundos), tornando-se inviável para $n > 24$. Isso ocorre porque a DP é obrigada a processar todos os subconjuntos de vértices, independentemente de haver ou não chance de um caminho existir.

7.2 Eficiência do Backtracking Instrumentado

O algoritmo de Backtracking, apesar de ter um pior caso teórico fatorial $O(n!)$, mostrou-se superior na prática para instâncias de até $n = 50$. Essa eficiência não foi por acaso, mas sim fruto das três camadas de otimização implementadas:

- **Filtragem Precoce:** Os `sanity checks` evitaram o início da recursão em grafos desconexos ou com muitos vértices de grau 1, resolvendo essas instâncias em tempo quase nulo ($0.0s$).

- **Efeito de Curto-Circuito:** Em grafos densos ($p = 0.5$), a abundância de soluções permitiu que a heurística de Warnsdorff encontrasse um caminho quase sem retrocessos (*backtracks*), mantendo o número de chamadas (`calls`) próximo do limite inferior n.

- **Inteligência de Poda:** No cenário crítico ($n = 30$ esparso), as podas de conectividade (`prunes_reach`) foram fundamentais. Os dados mostram que o algoritmo evitou entrar em centenas de ramos sem saída ao detectar, via BFS, que o subgrafo restante havia se tornado desconexo.

7.3 A Dificuldade na Transição de Fase

Aprendemos que a "dureza"do problema reside na **Transição de Fase**. Quando o grafo está na fronteira da conectividade ($n = 30$ e $p = 0.15$), ele possui arestas suficientes para não ser descartado pelos testes iniciais, mas não possui o suficiente para que um caminho seja encontrado facilmente. Foi nesse ponto que o esforço de busca explodiu para **941 chamadas**, exigindo o máximo das funções de poda para não resultar em *timeout*.

Em resumo, o trabalho demonstra que, para problemas NP-Completos em grafos, a escolha do algoritmo deve levar em conta a estrutura dos dados. O Backtracking instrumentado com podas baseadas na estrutura do grafo (graus e componentes conexos) provou ser a ferramenta mais robusta para lidar com a complexidade do Caminho Hamiltoniano em escala prática.

8 Referências Bibliográficas

Referências

[1] GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W. H. Freeman, 1979.

[2] KARP, R. M. *Reducibility Among Combinatorial Problems.* In: *Complexity of Computer Computations.* Springer, 1972.

[3] BELLMAN, R. *Dynamic programming treatment of the travelling salesman problem.* Journal of the ACM, 1962.

[4] CORMEN, T. H. et al. *Introduction to Algorithms.* 3rd ed. MIT Press, 2009.

[5] LIN, S.; KERNIGHAN, B. W. *An Effective Heuristic Algorithm for the Traveling-Salesman Problem.* Operations Research, 1973.